

Project Presentation Preparation Guide

Property Management Application using Power BI

Duppati Ashritha | 24WH1DB008

Under the guidance of Mr. Ch. Anil Kumar, Assistant Professor

Department of Information Technology, BVRIT Hyderabad College of Engineering for
Women

A slide-by-slide speaking script for viva, seminar, and project presentation

Slide 1 – Title

I would like to begin by introducing my project, titled Property Management Application using Power BI. This project has been carried out under the guidance of Mr. Ch. Anil Kumar, Assistant Professor in the Department of Information Technology at BVRIT Hyderabad College of Engineering for Women. The title itself reflects the two core pillars of this work, a complete property management platform and a dedicated business intelligence layer built using Power BI. I am presenting this today as a fully implemented and tested system, and over the next few slides I will walk you through the motivation, the design, the technology used, and the results we achieved.

Slide 2 – Table of Content

On this slide, I am giving you a quick overview of how my presentation is organised. I will start with the abstract and introduction to set the context, then move into the literature review and the research gaps that this project addresses. After that, I will cover the problem statement, objectives, the existing systems, and the proposed system, followed by the system architecture, the algorithms used, and the working modules. Towards the end, I will present the requirements, the technology stack, the experimental results, the result analysis, and finally the conclusion, future scope, and references. This structure allows me to move logically from why the project was needed to how it was built and what it achieved.

Slide 3 – Abstract

This slide summarises my entire project in a few lines. PropManage is a MERN stack platform, built using MongoDB, Express dot js, React dot js, and Node dot js, and it brings together property cataloguing, visit booking, payment handling, contract management, and maintenance tracking into a single system. The problem I am solving is that small and mid-size property operators today still depend on fragmented manual workflows such as spreadsheets, phone calls, and paper records. To address this, I designed a role-segmented, dual-portal system with real-time WebSocket alerts, HMAC verified payments, a multilingual voice-enabled AI assistant, and a Power BI analytics layer. I have validated this system thoroughly, with all thirty-eight

functional tests and all fourteen security tests passing successfully, which gives me strong confidence in what I am presenting today.

Slide 4 – Introduction

In this section, I want to explain the background and motivation behind choosing this project. Property seekers and small or mid-size operators still coordinate everything manually, through phone calls, messaging apps, and paper records, even though sectors like banking, hospitality, and e-commerce have already digitised years ago. This gap motivated me to build a platform that reduces administrative overhead while also improving accessibility, particularly for users who are more comfortable speaking in Hindi or Telugu rather than English. The real-world applications of this system are broad, ranging from rental and sale management firms to individual property owners, brokers, and self-service portals for tenants and buyers. This sets the stage for the more specific problem I will describe on the next slide.

Slide 5 – Problem Statement

Here, I am stating the core problem this project is trying to solve. There is no single system available today that covers the complete property lifecycle, starting from listing a property all the way to scheduling visits, collecting payments, managing contracts, and tracking maintenance. Because operators rely on manual and spreadsheet-based coordination, we see double-booked property visits, payment records that cannot be independently verified, and delayed staff awareness of new bookings due to inbox polling. On top of this, there is no assistance available for users outside of staff working hours, or for users who are only comfortable in a regional language, and there is no live, self-service business intelligence view for owners or administrators either. Each of these gaps is something I specifically designed PropManage to solve, and I will show you exactly how in the coming slides.

Slide 6 – Objectives

Building on the problem I just described, this slide lists the concrete objectives I set for myself while designing this system. My first goal was to unify cataloguing, scheduling, payments, contracts, and maintenance into one role-segmented, dual-portal platform. Alongside that, I wanted to deliver real-time, sub-second operational alerts to admin staff through WebSocket, so

that there is no delay caused by inbox polling, and to cryptographically verify every high-value payment using HMAC-SHA256 before committing any financial record. I also aimed to provide a context-grounded AI assistant with multilingual voice input and output in English, Hindi, and Telugu, and to enforce role-based access control so that a de-activated account loses access immediately, on its very next request. Finally, I wanted to expose a secure Power BI analytics API and empirically evaluate the whole system for correctness, security, and load performance.

Slide 7 – Scope of Work

On this slide, I am defining the boundary of what this project actually covers, so that the scope is clear before I go into the design details. Within scope, I have implemented property listing and search, visit and purchase booking, UPI and Razorpay payments, rental and purchase contracts, and maintenance tickets, along with real-time admin notifications, the multilingual voice AI assistant, Power BI analytics endpoints, and role-based security hardening. However, I have deliberately kept certain things out of scope for this phase, such as a native mobile application, full interface localisation with right-to-left layout, IoT or smart-lock integration, and predictive pricing analytics, all of which I revisit later under future scope. The three target user groups for this system are property seekers and tenants, property owners, and platform administrators. This clear scoping helped me focus on building a smaller set of features properly rather than a large set of features shallowly.

Slide 8 – Literature Review

For this project, I reviewed six papers that are directly relevant to the technical pillars of my system, and I have summarised them in this comparison table. The paper by Al-haimi and colleagues, along with the paper by Bhat and colleagues, confirm that property operators still rely on manual coordination and that the MERN stack's unified JSON pipeline reduces the effort of transforming data across layers. Meduri's work shows that large language model chatbots improve query resolution compared to older, rule-based bots, while Sabharwal and Sahni specifically identify the code-switching and low-resource barriers that voice assistants face in Indian languages, which is directly relevant to the multilingual assistant I built. I also reviewed Bowen and colleagues' security audit of JWT implementations, and Adediran and colleagues' study on AI adoption in property management, which together highlight that no existing system

combines these ideas into one deployed, evaluated platform. This is exactly the gap I explain in more detail on the next slide.

Slide 9 – Research Gaps

Based on the literature I just reviewed, I identified three clear research gaps that this project addresses. First, existing studies document workflow fragmentation in real estate and show that LLM chatbots improve query resolution, but they also flag issues like provider dependency and a lack of live-data grounding. Second, security research audits JWT and API vulnerabilities largely in isolation, as diagnostic tools, rather than as part of one integrated, adversarially tested system. Third, and most importantly, no existing study integrates verified payments, real-time notification, and conversational assistance into one deployed system, and no reviewed assistant is simultaneously domain-grounded, multilingual, and voice-enabled without added infrastructure cost. PropManage was built specifically to close all three of these gaps together, combining HMAC-verified payments, WebSocket notification, a multilingual voice AI assistant, and Power BI analytics into one empirically evaluated MERN platform.

Slide 10 – Existing Systems

Before describing my proposed system, I want to walk you through what is already available in the market today. Current approaches fall into three categories, purely manual coordination using phone calls, spreadsheets, and messaging apps, listing portals such as Bayut dot com and Dubizzle that only allow browsing without any booking, payment, or contract workflow, and commercial suites such as Buildium and AppFolio that cover more of the lifecycle but come at enterprise cost and still lack verified real-time payments or AI and voice assistance. Because of this, existing systems suffer from manual and time-consuming coordination, payment records that cannot be independently or cryptographically verified, delayed staff notification, no conversational or voice-based assistance, and high cost with poor scalability for small and mid-size operators. Recognising these specific weaknesses is what shaped the design of the proposed system I present next.

Slide 11 – Proposed System

This slide presents my proposed system as a direct answer to the weaknesses I just described. I designed a single, role-segmented, dual-portal MERN platform that covers the full property lifecycle end to end, and I added new features including real-time WebSocket staff notification, HMAC-SHA256 verified payments, a multilingual English, Hindi, and Telugu voice AI assistant, and a Power BI analytics API. As a result, the system becomes faster, since notifications are sub-second with no polling, more secure, because of cryptographic payment verification and per-request role-based access control, and more user-friendly and accessible, thanks to voice input and output in multiple languages, while also being scalable and intelligent through BI-ready analytics and context-grounded AI. The novelty I want to highlight here is a browser-native, zero-cost multilingual voice assistant integrated with a live-data-grounded chatbot, a combination that, based on my literature review, is not found together in any existing reviewed system.

Slide 12 – System Architecture

This diagram represents the complete flow of my system's architecture, and I want to walk you through it step by step. It begins with the user, who interacts through the frontend built in React dot js, then passes through the authentication layer, which uses JWT tokens and role-based access control, and from there can reach the voice assistant, which is powered by the Gemini API together with the browser's Web Speech API. Every request, whether it comes from a normal action or the voice assistant, flows into the backend APIs built with Node dot js and Express dot js, which in turn communicate with the MongoDB Atlas database. On the other side, the admin panel, which is the client-admin portal, and the Power BI reports and analytics layer both draw from this same backend and database, which means administrators and business intelligence dashboards are always working with the same live, consistent data rather than a separate, disconnected copy.

Slide 13 – Methodology / Algorithms

On this slide, I am explaining the five core mechanisms that actually make this system work, and I will go through each one briefly. The first is JWT stateless authentication with role-based access control, where the purpose is to verify identity without keeping server-side session state,

and the server signs a token at login and then re-fetches the user's role and active status on every protected request, so a blocked account loses access on its very next request. The second is HMAC-SHA256 payment verification, where the server independently recomputes the expected signature from the order and payment identifiers, guaranteeing that a recorded payment truly matches what the gateway processed and making it immune to client-side tampering. The third mechanism is real-time WebSocket notification, where the server broadcasts an event to the admin room immediately after the database saves it, without any polling interval at all. The fourth is the multilingual voice assistant pipeline, which takes input from the microphone through the browser's speech recognition, sends it to Gemini with a grounded and language-conditioned prompt, and streams the reply back through speech synthesis, and the fifth is the Power BI aggregation pipeline, which performs grouping, faceting, and time-bucketing directly inside MongoDB behind a dedicated API key, so it scales with reporting complexity rather than the number of records.

Slide 14 – Working Modules

This slide breaks the system down into seven working modules, and I find it easiest to explain them as a team of specialists, each responsible for one part of the system. The user module handles search and booking, the authentication module manages JWT tokens and role-based access control, and the voice assistant module, powered by Gemini and the speech API, handles all conversational and voice interactions. The booking and payment module, integrated with Razorpay, manages the entire transaction lifecycle, while the admin module takes care of approvals and CRUD operations for administrators. Underneath all of this sits the database module, built on MongoDB and Mongoose, and finally the reporting module, which exposes the Power BI API for analytics. Presenting the system this way helps show that each responsibility is cleanly separated, which makes the codebase easier to maintain and extend.

Slide 15 – Requirements

This slide lists the hardware and software needed to run PropManage, and as you can see, the requirements are quite modest. On the hardware side, a dual-core processor at two gigahertz or above is sufficient, along with a minimum of four gigabytes of RAM, though eight gigabytes is recommended, five hundred megabytes of free storage, and a stable broadband internet

connection. On the software side, the system runs on Windows, macOS, or Linux, using Node dot js eighteen LTS and React dot js eighteen point two, along with MongoDB Atlas in the cloud, VS Code as the development environment, and the Razorpay, Google Gemini, Cloudinary, and Mailjet APIs, plus Power BI Desktop or Service for the analytics side. I want to emphasise that every single one of these requirements is either open source or available on a free tier, which directly supports my claim that this system does not need an enterprise budget to run.

Slide 16 – Technology Stack

Here, I am presenting the complete technology stack organised into two groups, frontend and backend on one side, and third-party services and AI on the other. For the frontend and backend, I used React dot js eighteen for both the client-user and client-admin portals, Node dot js and Express dot js to build the REST API and business logic, MongoDB Atlas with Mongoose for document persistence, and Socket dot IO to power live admin notifications. For the third-party layer, Razorpay with HMAC-SHA256 handles verified purchase advances, the Gemini API combined with the Web Speech API delivers multilingual chat and voice input and output, Cloudinary and Mailjet manage media storage and OTP emails, and Power BI's REST API powers the live business intelligence dashboards. I chose each of these tools deliberately, based on the specific purpose it serves in this project, rather than picking a generic or unrelated stack.

Slide 17 – Experimental Results

This slide presents the actual experimental results I gathered while testing PropManage on a realistically scaled dataset of forty-five properties, one hundred and twenty users, two hundred and eighty bookings, one hundred and ninety-five payments, forty-one rental and twelve purchase contracts, and three hundred and ten financial plus eighty-seven service transactions. Out of this testing, all thirty-eight functional test scenarios passed successfully, and all fourteen adversarial security checks also passed, which gives strong confidence in both the correctness and the security of the system. The bar chart on this slide shows the endpoint latency in milliseconds across six key operations, login, properties, bookings, payments, dashboard, and AI chat, and you can clearly see that the AI chat endpoint is the slowest at around two thousand and eighty milliseconds, simply because it involves an external round trip to the Gemini API rather than any inefficiency in my own code. Under concurrent load, the system also sustained one

hundred simultaneous users at under five hundred milliseconds mean latency with less than zero point two percent errors, which validates that the system performs well even under realistic traffic conditions.

Slide 18 – Result Analysis

This slide consolidates all of my key results into a single, easy-to-read table. Functional correctness stands at thirty-eight out of thirty-eight scenarios passed, which is a full one hundred percent, and adversarial security tests also show fourteen out of fourteen passed, again one hundred percent. At one hundred concurrent users, the mean latency was four hundred and ninety-four milliseconds, with a ninety-fifth percentile of seven hundred and forty-three milliseconds, and the error rate at that load was only zero point one five percent, rising to three point one eight percent at two hundred users. Separately, through field observation, real-time notifications were consistently delivered to administrators in under two seconds. Taken together, these numbers directly confirm that every objective I set out to achieve at the beginning of this project has actually been met in practice, and not just claimed on paper.

Slide 19 – Conclusion

To conclude, I have successfully delivered a dual-portal, three-tier MERN platform that unifies property cataloguing, booking, payments, contracts, and maintenance into one coherent system. All five of my original objectives have been met, including real-time notification, HMAC-verified payments, role-based access control, the multilingual voice AI assistant, and Power BI analytics, and this has been empirically validated through thirty-eight out of thirty-eight functional tests, fourteen out of fourteen security tests, and stable performance at one hundred concurrent users with sub-five-hundred-millisecond latency. In practical terms, this system reduces administrative overhead, improves trust in payment handling, and adds voice accessibility in English, Hindi, and Telugu for users who need it. Most importantly, I want to highlight that this entire production-quality system was built using only free-tier tooling, which proves that small and mid-size property operators do not need an enterprise-level budget to benefit from a modern, secure, and intelligent platform like this one.

Slide 20 – Future Scope

While I am satisfied with what has been achieved so far, there are several directions in which this project can be extended further. On the AI side, I would like to migrate the current context-assembly grounding to full retrieval-augmented generation using MongoDB's vector search capability, and I also plan to build a native mobile application using React Native, with push notifications and camera-based evidence capture. Beyond that, I want to extend today's voice-level multilingual support into full interface localisation with a right-to-left layout for Arabic-speaking users, and introduce predictive analytics that can forecast rental pricing, vacancy periods, and late-payment risk from the booking and payment history the system already collects. I also intend to make the analytics aggregation more fault-tolerant by executing queries independently rather than as a single batch, and finally, I see real potential in integrating IoT devices such as smart locks and meters for keyless access and automated utility readings.

Slide 21 – References

This slide lists the first eight of my fifteen total references, each presented in proper IEEE citation format with the authors, title, journal, and year clearly stated. These references cover the digital transformation of the real estate industry, the architecture and benefits of the MERN stack, the impact of large language models on chatbot performance, and a systematic review of generative AI chatbots across domains. I have also cited important security literature, including the JWTKey study on cryptographic vulnerabilities in JWT applications, work on enhancing JWT authentication using behaviour history, the OWASP API Security Top Ten guidelines, and a comparative performance study between PostgreSQL and MongoDB. Every one of these sources is a real, peer-reviewed publication, and each one directly supports a specific design decision I made somewhere in this project, rather than being included just for the sake of having references.

Slide 22 – References (contd.)

This slide continues and completes my reference list from entry nine through fifteen. Among these, I want to specifically point out the study on AI adoption in property management transactions, which directly supports the research gap I identified earlier, along with two papers on retrieval-augmented generation, one focused on education chatbots and the other on vector

search using MongoDB Atlas, both of which justify the future direction I described for improving the AI assistant. I have also included a paper on performance optimisation techniques specific to MERN stack applications, which relates directly to the indexing and asynchronous processing techniques used in my own backend. Finally, the last two references, one on multilingualism in voice assistants and another on a working multi-language voice assistant using the Web Speech API, are the two sources most closely tied to the core novelty of my project, the multilingual voice assistant itself.

Slide 23 – Thank You

I would like to close my presentation here and thank the panel for taking the time to listen to me today. Through this project, I have tried to show that a small or mid-size property management business does not need an expensive, enterprise-grade system to get real-time notifications, verified payments, multilingual voice assistance, and live business intelligence dashboards. Every claim I have made in this presentation is backed by an actual working implementation and real test results, not just a theoretical design. I am happy to answer any questions you may have, and I welcome your feedback and suggestions on how this project can be improved further. Thank you once again for this opportunity.